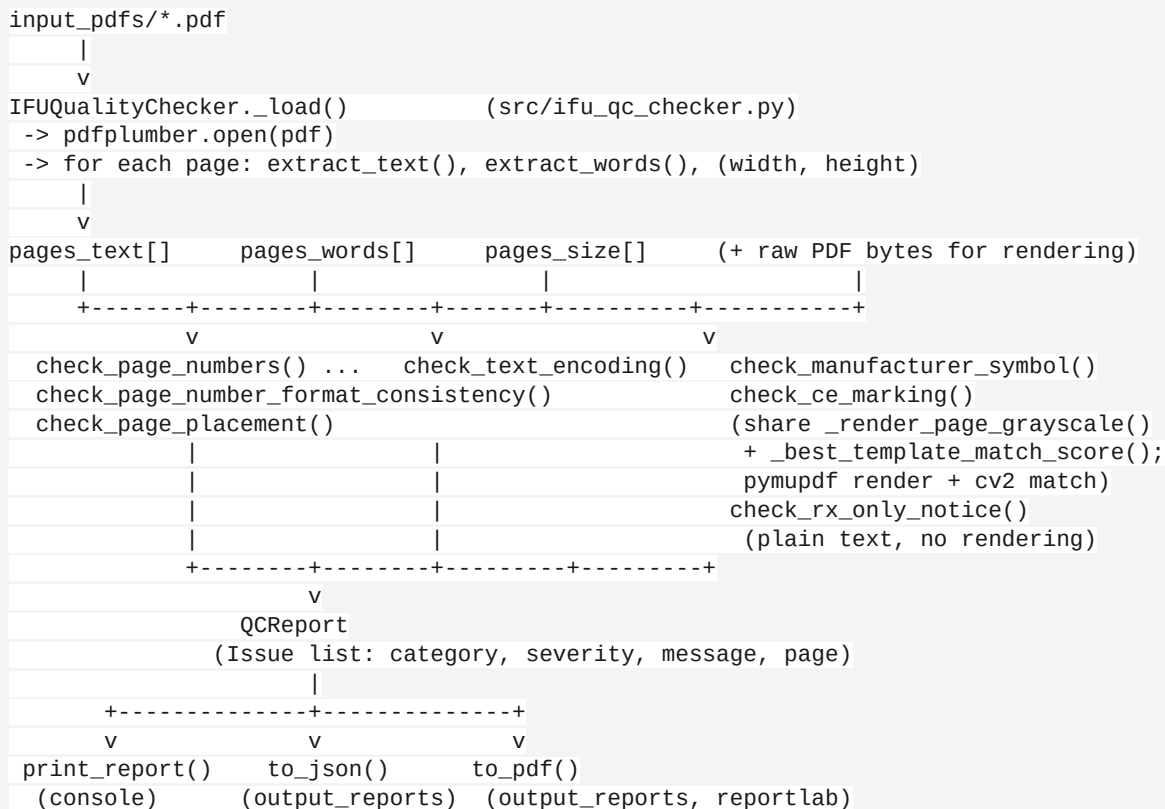


IFU QC Checker — Implementation Reference

This document explains *how* the checker works internally: the data it extracts from a PDF, the exact rules each check applies, the config keys that control them, and how the two hand-built test fixtures exercise those rules. For usage instructions, see [README.md](#).

1. Architecture overview



`IFUQualityChecker.__init__` loads the whole PDF once into three parallel per-page lists, and every check method reads from those — nothing re-opens the PDF. `run_all()` (bottom of `src/ifu_qc_checker.py`) is the entry point and simply calls each check in sequence:

```
def run_all(self):
    self.check_page_numbers()
    self.check_page_number_format_consistency()
    self.check_page_placement()
    self.check_text_encoding()
    self.check_manufacturer_symbol()
    self.check_ce_marking()
    self.check_rx_only_notice()
    return self.report
```

Every finding is a `QCReport.add(category, severity, message, page=None)` call. `severity` is `"FAIL"` (blocks pass) or `"WARN"` (informational). `QCReport.passed()` is simply `len(self.fails()) == 0`.

2. Check 1 — Page Number Verification

Category label in the report: "1. Page Number Verification".

2.1 Detection (`_detect_page_numbers`)

This is the shared engine every page-number rule reads from. It supports two document styles, chosen via `config["page_number_mode"]`:

"labeled" (default) — pages carry text like "Page 3 of 42". The full page text is matched against `config["page_number_pattern"]` (default `r"Page\s+(\d+)\s+of\s+(\d+)"`). Group 1 is the page number, group 2 is the declared total. The matching line is then located again in the word-coordinate data (`extract_words()`) so its on-page position (`x0`, `x1`, `top`) is known for the placement check.

"bare_number" — pages show just a standalone digit, no "Page"/"of" wording (common in compact IFUs). Because there's no surrounding text to anchor on, detection works purely from word coordinates, and supports two position rules via `config["bare_number_position_rule"]`:

"footer_zone" (default) — the number must sit within the bottom `footer_zone_ratio` slice of the physical page (default bottom 15%, i.e. `top >= height * 0.85`); the candidate closest to a page **edge** wins, since a real page number sits near the margin, not mid-footer.

```
digit_re = re.compile(r"\d{1," + str(max_digits) + "}")
candidates = [
    w for w in words
    if digit_re.fullmatch(w["text"])
    and w["top"] >= page_height * footer_zone_ratio
]
best = min(candidates, key=lambda w: min(w["x0"], page_width - w["x1"]))
```

"last_on_page" — for documents with no dedicated footer margin, where the number just follows wherever the body content happens to end (which varies page to page). Instead of a fixed bottom-of-page zone, whichever digit sits lowest (`max(top)`) on the page is taken as the number, with edge-proximity only used to break ties on the same line. Pages listed in `unnumbered_pages` are **skipped entirely** under this rule (not just exempted from the "missing" check) — cover/back-cover pages routinely carry their own trailing digits (addresses, dates, catalog/CE numbers), and without a footer zone to filter by, those would otherwise be misdetected as the page number. This was caught empirically: an early version searched every page and picked up a stray "0123" from CE-mark text on an exempted back-cover page, producing several false FAILs (duplicate/position-mismatch/placement) that had nothing to do with actual page numbering.

`bare_number_max_digits` (default 4) bounds how many digits count under either rule, to avoid matching unrelated numbers.

Both modes populate the same shape: `{idx: {"num", "raw", "total"?, "x0", "x1", "top", "page_width", "page_height"}}`, cached on `self._page_number_info` so repeated checks don't redo the work.

2.2 `check_page_numbers()` — the core rules

Runs, per document, in this order:

Rule	Logic	Severity
Missing number	Page not in <code>info</code> and not in the resolved <code>unnumbered_pages</code> set	FAIL
Declared total mismatch	("labeled" mode only) <code>d["total"] != total_pages</code>	FAIL

Duplicate number	Same <code>num</code> detected on 2+ physical pages	FAIL
Missing in sequence	<code>expected_seq - declared_nums</code> is non-empty	FAIL
Out-of-sequence	Declared numbers, read in physical page order, aren't non-decreasing	FAIL
Bare-number position mismatch	("bare_number" mode only) <code>d["num"] != idx</code> — the printed number must equal the page's actual physical position	FAIL

`unnumbered_pages` (config) accepts "first", "last", or explicit 1-indexed ints — resolved once per run via `_resolve_unnumbered_pages()`. This is how a cover page or an unnumbered divider page is exempted from the "missing number" rule.

Example real output (from `input_pdfs/test_scenario_page_placement.pdf`, an 8-page fixture with page 6 left blank, page 7 misprinted as "5" instead of "7"):

```
[FAIL] No page number found in the footer area (page 6)
[FAIL] Duplicate page number '5' found on physical pages [5, 7]
[FAIL] Missing page number(s): [6, 7]
[FAIL] Printed page number '5' does not match its physical position in the document (page 7)
```

2.3 `check_page_number_format_consistency()`

Independent of the numeric rules above — this checks *formatting*, not value. Each page's raw matched text (e.g. "Page 3 of 42") is turned into a template by substituting the number (and total, if present) with placeholders: "Page {N} of {T}". The most common template across the document becomes the majority; any page whose template differs (different capitalization, spacing, punctuation, leading zeros, wording) is flagged — even though it still matches the loose regex.

2.4 `check_page_placement()`

Gated by `config["enforce_left_right_placement"]` (default `True` in this project's `config.py`, `False` if unset). For every detected page number:

1. If `bare_number_position_rule != "last_on_page"` and `top < page_height * footer_zone_ratio` -> FAIL, "not positioned in the footer/below area." This sub-check is skipped entirely under "last_on_page": detection there has already taken whatever sits lowest on the page as *the* number, so re-applying a fixed-zone requirement would just contradict the reason that rule exists (documents with no dedicated footer, where content length varies) and produce the same false FAILs the rule was built to avoid.
2. Compute `center_x = (x0 + x1) / 2`. `is_right_half = center_x > page_width / 2`.
3. `expects_right = (declared_num % 2 == 1)` — **odd pages must be right-aligned in the footer, even pages left-aligned**. This side rule still applies under both position rules.
4. Mismatch -> FAIL with the specific side that was expected vs. found.

Note this uses the *declared* page number, not the physical page index — so a misprinted number (like the duplicate "5" on physical page 7 above) is judged by what's printed, not where it physically sits, which is why that fixture's page 7 didn't also fail placement: it printed "5" (odd) and was correctly right-aligned.

3. Check 2 — Text Encoding / Garbled Text Verification

Category label in the report: "2. Text Encoding / Garbled Text".

3.1 Why this check exists

A PDF's visible glyphs and its underlying character codes are two separate things, connected by the font's encoding table (and optionally a [ToUnicode](#) CMap). When a font is swapped or embedded incorrectly, that mapping can break: the page can still *render* something, but the codes no longer resolve to the intended Unicode characters. Text-extraction tools ([pdfplumber](#), built on [pdfminer.six](#)) then surface whatever those codes *do* resolve to — typically the Unicode replacement character (■), Private-Use-Area glyphs from an icon font, or literal ? substitutions from encoders that can't represent a character at all. This check inspects `pages_text` (the same extracted text every other check uses) for exactly those symptoms.

3.2 The three rules (`check_text_encoding`)

```
_GARBLED_CHAR_RE = re.compile(
    "[\uFFFF\u2500-\u25FF\uE000-\uF8FF"
    "\x00-\x08\x0B\x0C\x0E-\x1F]"
)
_REPEATED_SYMBOL_RE = re.compile(r"([?<>#%^\*~`|\\@])\1{2,}")
```

Rule	Trigger	Severity	Rationale
Undefined character	Any char matching <code>_GARBLED_CHAR_RE</code> : <code>U+FFFD</code> (replacement char), <code>U+E000-U+F8FF</code> (Private Use Area), <code>U+2500-U+25FF</code> (box-drawing / geometric shapes), or a stray C0 control byte	FAIL	These are unambiguous — normal IFU prose never legitimately contains them
Repeated symbol run	3+ of the same ASCII junk symbol in a row (<code>???</code> , <code><<<</code> , <code>###</code> , ...) via <code>_REPEATED_SYMBOL_RE</code>	WARN	A common <i>visual</i> signature of font-substitution garbage, but not impossible in legitimate content, so kept as a warning
High junk-symbol density	<code>text.count('?') + text.count('<') + text.count('>') >= garbled_symbol_warn_threshold</code> (default 5)	WARN	A single <code><</code> or <code>></code> is often legitimate (" <code><50°C</code> "); many on one page usually isn't

The FAIL rule is deliberately narrow (only characters with no legitimate use in body text) so it never fires on real content; the two WARN rules are density/pattern heuristics, so they stay advisory.

Real output from [input_pdfs/test_scenario_undefined_text.pdf](#):

```
[FAIL] Unreadable/undefined character(s) found in extracted text – likely a font/encoding issue (e.g. a font was swapped and its character codes no longer map to the correct
```

```
glyphs): '\ue001', '\ue002', '\ue003', '■' (page 2)
[WARN] Repeated symbol run(s) found (3+ in a row) for character(s) <, ? – may indicate
garbled text from a font substitution issue (page 3)
[WARN] Unusually high count of '?' / '<' / '>' characters (11) – verify this page isn't
rendering garbled text due to a font/encoding problem (page 4)
```

4. Check 3 — Manufacturer Symbol Verification

Category label in the report: "3. Manufacturer Symbol".

4.1 Why this needs real image detection

The other two checks work entirely on `pages_text/pages_words` — text pdfplumber already extracted. This one can't: the required ISO 7000-3082 "Manufacturer" pictogram (a factory silhouette — three roof peaks merging into a chimney, see `assets/manufacturer_symbol_template.png`) is graphics, and may be drawn as a raster image or vector art with **no accompanying text at all**. There is nothing in `pages_text` to search. So `check_manufacturer_symbol()` renders the target page to an actual image and does image template matching, rather than reusing the text-extraction pipeline.

4.2 Algorithm (`check_manufacturer_symbol`)

The page-resolution and rendering machinery here is factored into three shared helpers reused by `check_ce_marking()` too (§5): `_resolve_target_page_index(location)`, `_render_page_grayscale(pymupdf, cv2, np, target_idx, dpi)`, and `_best_template_match_score(cv2, page_gray, template, width_ratios)`.

Gated by `config["require_manufacturer_symbol"]`. Steps:

1. Import `pymupdf`, `cv2` (OpenCV), `numpy` — all optional. If any is missing, add a **WARN** ("install pymupdf, opencv-python and numpy") and return; the check never hard-fails the run over a missing dependency.
2. Resolve the target page from `config["manufacturer_symbol_page"]` ("last" by default, also accepts "first" or a 1-indexed int).
3. Load the reference template — `config["manufacturer_symbol_template"]`, or the bundled `assets/manufacturer_symbol_template.png` if unset — as grayscale via `cv2.imread`. Missing file -> WARN and return.
4. Render just the target page to a raster image with `pymupdf` at `config["manufacturer_symbol_render_dpi"]` (default 150 DPI), converted to grayscale.
5. **Multi-scale template match**: the symbol's size relative to the page isn't known ahead of time, so the template is resized to several candidate widths — `(0.07, 0.09, 0.12, 0.16, 0.20, 0.25)` × page width — and matched at each via `cv2.matchTemplate(..., cv2.TM_CCOEFF_NORMED)`. The highest score across all scales wins.

```
for width_ratio in (0.07, 0.09, 0.12, 0.16, 0.20, 0.25):
    resized_w = max(10, int(page_w * width_ratio))
    resized_h = max(10, int(template_h * (resized_w / template_w)))
    resized = cv2.resize(template, (resized_w, resized_h), interpolation=cv2.INTER_AREA)
    result = cv2.matchTemplate(page_gray, resized, cv2.TM_CCOEFF_NORMED)
    _, max_val, _, _ = cv2.minMaxLoc(result)
    best_score = max(best_score, max_val)
```

6. `best_score < config["manufacturer_symbol_match_threshold"]` (default 0.6) -> **FAIL**.

Why scales below ~0.06 are excluded: at that size the resized template is only a few dozen pixels wide, and correlation against a mostly-blank page background scores misleadingly high regardless of whether the symbol is actually present — this was caught empirically (see below), not assumed.

4.3 Threshold calibration

The 0.6 threshold and the excluded small-scale range were both derived by measuring real match scores, not guessed. Two fixtures (§6) were built — one with the symbol embedded on the last page, one without — and matched at every scale:

width ratio	with symbol	without symbol
0.07	0.642	0.462
0.09	0.847	0.372
0.12	0.804	0.320
0.16	0.574	0.218
0.20	0.402	0.155
0.25	0.303	0.131

The first attempt also included ratios `0.03/0.05`: those scored `0.588/0.568` on the *empty* page — high enough to falsely clear a 0.55 threshold — because at ~40px wide the template is mostly white margin, and white-on-white correlates well no matter what's underneath. Dropping those two scales and raising the threshold to 0.6 gives a clean, wide margin between the two fixtures (0.847 vs. 0.462) instead of a near-miss.

5. Check 4 — CE Marking Verification

Category label in the report: `"4. CE Marking"`.

5.1 Why two templates, and why "either is fine"

The CE conformity mark has two legitimate on-page forms depending on device classification: the bare mark, or the mark followed by the notified body's 4-digit ID (e.g. `CE0344`) when the device's conformity assessment required notified-body involvement. A document with either form is compliant — flagging a document for using the "wrong" one would be a false positive — so `check_ce_marking()` accepts a match against **any** of `config["ce_marking_templates"]` (default: `assets/ce_mark_template.png` and `assets/ce_mark_0344_template.png`), taking the max score across all of them, all at the same multi-scale search used for the manufacturer symbol.

Both reference images were built the same way as the manufacturer symbol's — a faithful geometric recreation (two rounded "C"/mirrored-"C"-with-bar shapes forming the CE glyph pair, per the standard mark's construction) via `PIL.ImageDraw`, not a scanned/sourced logo asset.

5.2 Algorithm (`check_ce_marking`)

Same shape as `check_manufacturer_symbol()`, reusing the shared helpers from §4.2, with two differences: multiple templates are tried (`max()` over each template's own best score), and the search uses a different scale range — `(0.05, 0.07, 0.09, 0.12, 0.15, 0.20) × page width` — since the CE mark tends to render smaller

on-page than the manufacturer pictogram. `best_score < config["ce_marking_match_threshold"]` (default `0.6`) -> **FAIL**. Same WARN-and-skip behavior if `pymupdf/cv2/numpy` are missing or no template image loads.

5.3 Threshold calibration

Measured against three fixtures (§7) — bare CE embedded, CE0344 embedded, and neither:

width ratio	<i>bare CE</i> fixture, <code>ce_only</code> template	<i>CE0344</i> fixture, <code>ce_only</code> template	<i>CE0344</i> fixture, <code>ce_0344</code> template	<i>missing</i> fixture, best of both
0.05	0.313	0.313	0.351	0.351
0.07	0.439	0.934	0.276	0.243
0.09	0.696	0.354	0.585	0.235
0.12	0.209	0.213	0.507	0.213

Two things stand out: the *CE0344* fixture's strongest match is actually against the **bare-CE** template (0.934) at a smaller scale — because the CE0344 artwork visually contains the bare-CE glyph pair as its left portion, so a smaller `ce_only` template correlates against just that sub-region. That's fine: since the check takes the max across all templates and scales, this only ever makes detection *more* confident, never less. The *missing* fixture tops out at 0.351 regardless of template — a wide margin below the 0.6 threshold, so no recalibration was needed here (unlike the manufacturer symbol, which required excluding small scales — see §4.3).

6. Check 5 — Prescription (Rx Only) Notice Verification

Category label in the report: `"5. Prescription (Rx Only) Notice"`.

Unlike the previous two, this notice is ordinary printed text (see the "Rx Only" line in the reference photo), so `check_rx_only_notice()` doesn't render anything — it reuses `pages_text` like Check 1 and Check 2. Gated by `config["require_rx_only_text"]`. Logic:

```
target_idx = self._resolve_target_page_index(self.config.get("rx_only_page", "last"))
expected = self.config.get("rx_only_text", "Rx Only")
target_text = re.sub(r"\s+", " ", self.pages_text[target_idx - 1]).strip().lower()
if expected.lower() not in target_text:
    # FAIL
```

A simple case-insensitive substring match against the resolved target page ("`last`" by default) — no fuzzy matching or regex, since this is a fixed, short, standardized phrase rather than free-form prose.

7. Report outputs

`QCReport` (dataclass, `src/ifu_qc_checker.py`) holds a flat list of `Issue(category, severity, message, page)` and renders it three ways:

- `print_report()` — grouped by category, [FAIL] / [WARN] markers, printed to console by `run.py`.
- `to_json(path)` — `{source_file, generated_at, result, fail_count, warn_count, issues[]}`, written to `output_reports/<name>.qc_report.json`. Machine-readable, meant for CI.
- `to_pdf(path)` — a formatted PDF (via `reportlab`) with a PASS/FAIL summary table and one color-coded issues table per category, written to `output_reports/<name>.qc_report.pdf`.

`run.py` exits non-zero if any processed PDF has `report.passed() == False`, so it can gate a CI pipeline directly.

8. Configuration reference (`config.py`)

Key	Code default (if key omitted)	This project's <code>config.py</code> value	Used by
<code>page_number_mode</code>	"labeled"	"bare_number"	<code>_detect_page_numbers</code>
<code>page_number_pattern</code>	(required in labeled mode)	<code>r"Page\s+(\d+)\s+of\s+(_detect_page_numbers</code> (labeled mode only)	
<code>bare_number_max_digits</code>	4	4	<code>_detect_page_numbers</code> (bare mode only)
<code>bare_number_position_rule</code>	"footer_zone"	"last_on_page"	<code>_detect_page_numbers</code> + <code>check_page_placement</code> (bare mode only)
<code>unnumbered_pages</code>	[]	["first", "last"]	<code>_resolve_unnumbered_pages</code>
<code>enforce_left_right_placement</code>	False	True	<code>check_page_placement</code> gate
<code>footer_zone_ratio</code>	0.85	0.85	bare-number detection + placement footer check
<code>garbled_symbol_warn_threshold</code>		5	<code>check_text_encoding</code> high-density rule
<code>require_manufacturer_symbol</code>	False	True	<code>check_manufacturer_symbol</code> gate
<code>manufacturer_symbol_page</code>	"last"	"last"	<code>check_manufacturer_symbol</code> target page
<code>manufacturer_symbol_template</code>	(bundled asset)	None -> bundled asset	<code>check_manufacturer_symbol</code> reference image
<code>manufacturer_symbol_render_dpi</code>	150dpi	150	page-render resolution
<code>manufacturer_symbol_match_threshold</code>	0.6	0.6	minimum match confidence
<code>require_ce_marking</code>	False	True	<code>check_ce_marking</code> gate
<code>ce_marking_page</code>	"last"	"last"	<code>check_ce_marking</code> target page

<code>ce_marking_templates</code>	<i>(bundled assets)</i>	None -> both bundled assets	<code>check_ce_marking</code> reference images (any match passes)
<code>ce_marking_render_dpi</code>	150	150	page-render resolution
<code>ce_marking_match_thresho</code>	0.6	0.6	minimum match confidence
<code>require_rx_only_text</code>	False	True	<code>check_rx_only_notice</code> gate
<code>rx_only_page</code>	"last"	"last"	<code>check_rx_only_notice</code> target page
<code>rx_only_text</code>	"Rx Only"	"Rx Only"	expected notice text (case-insensitive)

9. Test fixtures

All fixtures live in `input_pdfs/` and were generated by one-off scripts (not committed — `input_pdfs/*.pdf` is gitignored) to exercise real extraction/rendering behavior, not just unit-test the regex or matching logic in isolation.

`test_scenario_page_placement.pdf` (8 pages)

Built with `reportlab.pdfgen.canvas`. Planted defects, one per rule:

Page	Defect	Rule exercised
1	Cover, no number	exempted via <code>unnumbered_pages: ["first", ...]</code>
2	"2" in footer, left	correct even/left — no finding
3	"3" in footer, right	correct odd/right — no finding
4	"4" in footer, right	placement FAIL (even page, wrong side)
5	"5" in footer, right	correct — no finding
6	no footer number at all	missing-number FAIL
7	"5" printed again (should be "7")	duplicate FAIL + bare-number position-mismatch FAIL + missing-"7" FAIL
8	back cover, no number	exempted via <code>unnumbered_pages: [..., "last"]</code>

test_scenario_no_footer.pdf (4 pages)

Built to exercise bare_number_position_rule: "last_on_page" specifically — a layout with no dedicated footer margin, where the number just follows wherever content ends:

Page	Layout	Purpose
1	Cover, no number	exempted via unnumbered_pages: ["first", ...]
2	Two lines of body text, number placed right after (~24% down the page, nowhere near the bottom edge)	proves detection isn't tied to a fixed bottom-of-page zone
3	45 lines filling the page, number near the bottom, right-aligned	shows the "page is full" case is handled the same way
4	Back cover with a "Catalog No. 9081" line and no page number	proves the exempted page is skipped entirely rather than misreading "9081" as a page number

Run against both rules for comparison — check_page_numbers() + check_page_placement() only, isolated from the other checks:

```
--- last_on_page (new) ---
(no page-number findings)
--- footer_zone (old) ---
FAIL: No page number found in the footer area
FAIL: Missing page number(s): [2]
```

Page 2's number sits at roughly top / page_height ≈ 0.24 — well short of the footer_zone_ratio default of 0.85 — so the old rule genuinely can't see it; the new rule finds it correctly since it only cares about relative vertical position on the page, not proximity to the physical edge.

test_scenario_undefined_text.pdf (4 pages)

This one required a real font/encoding defect, not a fabricated string, since the whole point of Check 2 is testing what pdfplumber actually extracts from a broken font mapping. reportlab can't produce that directly (it only draws valid, correctly-mapped text), so this PDF was hand-assembled at the raw PDF object level:

```
<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica
/Encoding << /Type /Encoding /BaseEncoding /WinAnsiEncoding
/Differences [1 /uniFFFF /uniE001 /uniE002 /uniE003] >> >>
```

Page 2's content stream draws the raw byte codes <01020304> using this font. Per the Adobe Glyph List naming convention, pdfminer.six (which pdfplumber sits on) resolves glyph names of the form uniXXXX straight back to that Unicode code point when there's no ToUnicode CMap — so extract_text() on that page genuinely returns ■■■■, exactly reproducing what happens when a real document's font gets swapped and its character codes desync from the intended glyphs.

Page	Content	Rule exercised
------	---------	----------------

1	Clean text incl. a legitimate <code><50C</code>	control — confirms no false positives
2	Custom-font bytes -> real <code>■</code> /PUA extraction	undefined-character FAIL
3	Literal <code>???</code> / <code><<<</code> in plain text	repeated-symbol WARN (+ high-density WARN, expected overlap)
4	Many scattered <code>?/</></code>	high-density WARN only

All pages in both fixtures carry correctly-placed footer numbers, so every finding in each report is attributable only to the check under test — the Page Number check passes cleanly on the encoding fixture, and vice versa.

`test_scenario_manufacturer_symbol_present.pdf` / `...missing.pdf` (2 pages each)

A minimal pair used to calibrate and verify Check 3 in isolation. Both have an identical page 1 (ordinary body text) and page 2 (manufacturer name/address text). The *present* variant additionally draws `assets/manufacturer_symbol_template.png` onto page 2 at 10% of page width via `reportlab's drawImage`; the *missing* variant doesn't. Running `check_manufacturer_symbol()` against each: *present* -> no finding (best match 0.847, ≥ 0.6 threshold); *missing* -> FAIL (best match 0.462). See §4.3 for the full per-scale score table these fixtures produced, which is what the threshold and excluded-scale-range were tuned against.

`test_scenario_ce_only.pdf` / `test_scenario_ce_0344.pdf` / `test_scenario_ce_missing.pdf` (2 pages each)

Three variants used to calibrate and verify Checks 4 and 5 together: page 2 of each carries manufacturer name text and, in the first two, one of the two CE reference images (*ce_only* at 8% page width, *ce_0344* at 10%) plus an "Rx Only" line; the *missing* variant has neither. `check_ce_marking()` + `check_rx_only_notice()` against each: *ce_only* -> no findings; *ce_0344* -> no findings; *missing* -> CE-marking FAIL (best match 0.351) **and** Rx-Only FAIL. See §5.3 for the full score table.

10. Extending the checker

To add a new check:

1. Write a `check_*(self)` method on `IFUQualityChecker` that reads from `self.pages_text` / `self.pages_words` / `self.pages_size` and calls `self.report.add(category, severity, message, page=...)` for each finding.
2. Add any tunables to `CONFIG` in `config.py` with a comment explaining them, read via `self.config.get("key", default)`.
3. Call the new method from `run_all()`.
4. Document it in `README.md`'s numbered checklist and configuration section.